

1 Skewed Dataset Generation [EXPERIMENT 0].

To evaluate the system performance under non-uniform loads, I implemented an advanced skew-generation strategy capable of creating a controlled workload imbalance without resorting to simple key duplication, while maintaining the exact same partitioning function outlined in Module 1. The algorithm is governed by three main parameters: a skewness fraction $\sigma \in [0, 1]$, a size $M \ll P$ of the target subset hosting the skewness traffic, and a parameter $\theta \geq 1.0$ that controls the steepness of the traffic spike. To determine whether a tuple belongs to the skewed portion, a uniform random value $u \sim \mathcal{U}(0, 1)$ is drawn. If $u < \sigma$, the tuple is assigned to the hotspots, so that approximately $\sigma \cdot 100\%$ of the data is skewed. To avoid hotspot concentration in the first physical partitions, P_{target} is not chosen directly in $[0, M - 1]$: an indirection layer retrieves the true P_{target} from a pre-shuffled set of M distinct partitions, distributing the load across the entire system.

To achieve realistic, Zipfian-like traffic peaks rather than an artificial flat plateau, this lookup index is calculated by applying a power-law distortion to a uniform random variable (u_{idx}^θ). This mathematical compression forces the generated indices to gravitate toward the 0-th element of the subset, dynamically sculpting the hotspot severity based on θ .

To obtain realistic Zipfian peaks instead of a uniform plateau, the index is generated by applying a power-law distortion to a uniform variable (u_{idx}^θ). This concentrates the probability on the first elements of a subset of M target partitions (shortlist of hotspots), making the indices fall more often on the first partitions and modulating the hotspot severity via θ . The remaining uniform fraction $1 - \sigma$ is instead uniformly distributed over the entire global domain K_{max} . For the remaining uniform fraction $1 - \sigma$, keys are simply generated uniformly across the global K_{max} domain.

Once the target partition P_{target} is selected, we construct a 64-bit key that deterministically maps to it while respecting the global constraint K_{max} , leveraging the partition function $P(k) = (k \oplus (k \gg 32)) \& (P - 1)$. The construction splits into two regimes. For $K_{\text{max}} < 2^{32}$, we have $k_{\text{high}} = 0$, so the XOR becomes neutral and the hash collapses to $P(k) \approx k \bmod P$. We introduce a degree of freedom via R , which enumerates valid multiples of the partition and preserves high cardinality while maintaining determinism: $k_{\text{low}} = R \cdot P + P_{\text{target}}$, thus enforcing $P(k) = P_{\text{target}}$ by construction.

For $K_{\text{max}} \geq 2^{32}$, the upper half contributes to the hash through XOR, introducing interference $O_{\text{high}} = k_{\text{high}} \bmod P$. This effect is pre-compensated in the lower bits via a corrected offset $B = P_{\text{target}} \oplus O_{\text{high}}$, which cancels the XOR-induced disturbance during partitioning. The final key is assembled as $k = (k_{\text{high}} \ll 32) \mid k_{\text{low}}$, ensuring deterministic routing to P_{target} in both regimes.

When the hash function is evaluated, both regimes collapse to the same outcome: in Case A the construction enforces a pure residue class, while in Case B the XOR-induced high-bit interference is exactly canceled by the pre-embedded compensation term, so that in both cases $P(k) = P_{\text{target}}$ holds deterministically. An additional appendix PDF [Skewed_data_generation.pdf] with purely graphical explanations is provided to offer a more intuitive view of the mechanism. While not required for understanding the generation process, it complements the present formulation. Full algebraic derivations are omitted due to space constraints, as they would require a substantially longer treatment.

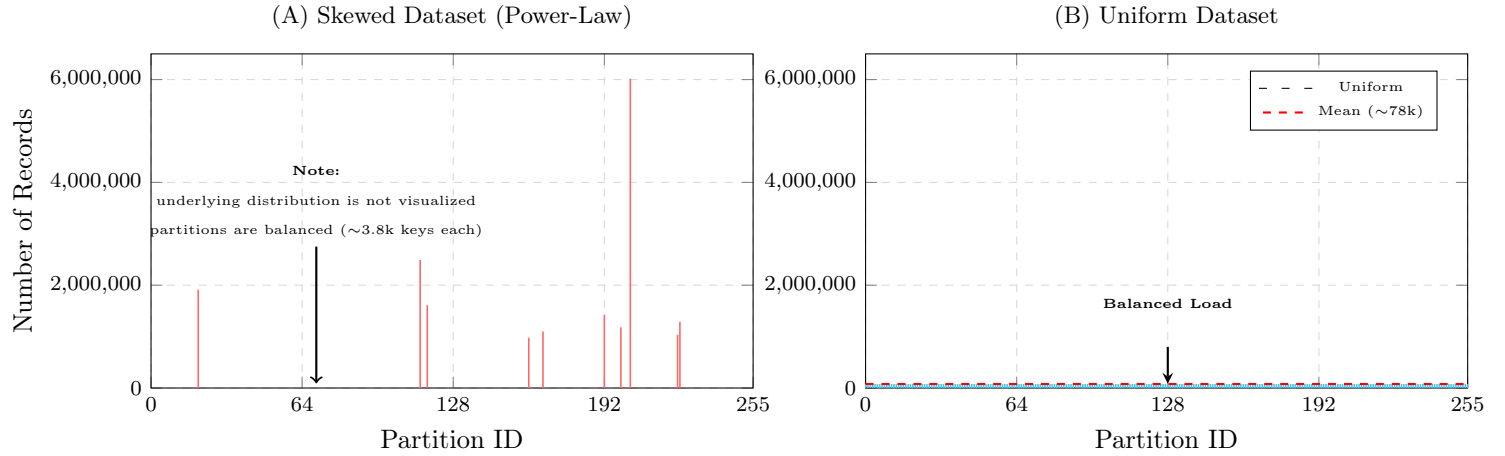


Figure 1: These two images show the distributions of the keys on the partitions under the two reference assumptions of this module, uniform and asymmetric, used in the tests that will follow; note that I decided to show only the distribution of the relation R . As for the configuration, we have 256 partitions, 2×10^7 records, $K_{\text{max}} = 10^7$ and for the uniform assumption a $\sigma = 0$, while for the asymmetric assumption $\sigma = 0.95$, subset = 10, $\theta = 2$.

2 strategy LoopParallel

2.1 OpenMP Loop-Level Parallelism

The parallelization strategy is consistent with the native C++ concurrent implementation developed in Module 2. The main difference concerns workload balancing in the local join phase. In the native C++ implementation, a pseudo-uniform distribution was assumed; however, in this module, consistency is required for both uniform and skewed datasets.

Compute Histogram Kernel. As already discussed, the histogram kernel is not affected by the data distribution across partitions, since each iteration performs uniform work of complexity $O(1)$ (computing the partition ID and updating the counter). Therefore, a static scheduling policy is sufficient, ensuring a balanced distribution with minimal overhead.

Parallelization uses private local histograms per thread to avoid contention. The parallel region is defined with `#pragma omp parallel` and `default(none)`, where The key set, the partition parameter p , and the histogram container are shared: rel is read-only, p is constant, and local $hists$ is thread-safely indexed. Each thread uses `omp_get_thread_num()` to access its dedicated histogram row, ensuring disjoint memory access.

The work distribution is therefore static, `#pragma omp for schedule(static)`.

Scatter Kernel. Again, parallelization mirrors the native C++ implementation, using private per-thread buffers (`local_buffer`) and counters (`buffer_counts`) to reduce contention. The parallel region is defined with `#pragma omp parallel` and `default(none)`.

The `rel` key set, `out` output array, p partition parameter, and `thread_offsets` are declared shared: `rel` is read-only, p is constant, and `out` is written to disjoint memory regions per thread. Each thread uses `omp_get_thread_num()` to initialize a private `my_next` array, ensuring that writes stay within the assigned bounds.

Work distribution uses `#pragma omp for schedule(static)`, ensuring an even partition among threads.

Local Join Kernel. Unlike histogram and scatter, the local join phase is highly sensitive to data skew. Since the cost of processing a partition depends on its cardinality, asymmetric datasets may produce severe load imbalance if partitions are assigned statically to threads. For this reason, the work-sharing loop is configured with `#pragma omp for schedule(runtime)`, allowing the scheduling policy (`static`, `dynamic`, `guided`) to be selected at runtime and evaluated experimentally without recompilation.

The parallel region is defined with `default(none)`, explicitly declaring shared read-only structures (`Rpart`, `Spart`) and the shared output arrays (`thread_results`, `thread_timings`). To avoid synchronization overhead during the join itself, each thread accumulates results in strictly local variables and writes them only once at the end into a dedicated slot indexed by `omp_get_thread_num()`. Since these writes are completely non-overlapping, the loop safely uses the `nowait` clause, removing the implicit barrier and reducing unnecessary synchronization costs.

Local Join Kernel: Memory Management under Skew. [EXPERIMENT 1.1] In the native concurrent C++ implementation developed in Module 2 (see Report, Section 1.4.3 for details), an allocation hoisting optimization is applied under the assumption of uniform partition distribution. Each thread maintains a single local `ska::flat_hash_map`, allocated outside the partition loop, initialized once via `reserve()` and reused across partitions through `clear()`, thereby reducing allocation and rehashing overhead in balanced workloads.

However, experimental results on unbalanced datasets show that this strategy becomes inefficient with unbalanced partition sizes. The problem is that the map’s capacity grows based on the largest partition seen by a thread. When a large partition is processed, repeated rehashing increases the internal bucket array to a size that satisfies the load factor, typically well beyond the initial estimate of `reserve((R.size() / p) * 2)` (which assumes approximately uniform partition sizes).

Once this oversized capacity is reached, processing a subsequent small partition becomes expensive: `clear()` does not depend on the number of stored elements, but on the total size of the data structure. Consequently, the cost becomes proportional to the map’s maximum historical capacity, not the active workload.

As part of the analysis, three implementations were evaluated. A basic solution where I have a new hash map for each partition. The solution just discussed where I have a single hash map per thread with hoisting.

And finally, a new solution in which I implemented an adaptive reset strategy. The idea is that before processing each partition, the required capacity is estimated as `needed_buckets = 2 * |Rp|` (target load factor 0.5) and compared with `bucket_count()`.

If the current map is oversized, it is replaced via `swap()` with a newly allocated map of the correct scale, ensuring excess memory release with $O(1)$ complexity. Otherwise, the existing map is reused and only flushed, with a cost proportional to the current partition size rather than the historical peak. This hybrid approach seeks to avoid both excessive growth and unnecessary reallocations.

Memory Strategy	Dataset	Build (ms)	Probe (ms)	Total Join (ms)
Hoisted (1 map/thread)	Uniform	53.00 ± 1.94	53.03 ± 8.25	155.93 ± 1.98
	Skewed	81.77 ± 4.15	94.54 ± 16.21	741.43 ± 72.37
1 map/partition	Uniform	86.72 ± 1.89	67.52 ± 17.66	224.23 ± 1.77
	Skewed	75.95 ± 2.95	71.34 ± 14.15	414.72 ± 18.03
Adaptive Memory	Uniform	56.31 ± 7.38	57.28 ± 8.80	178.22 ± 2.61
	Skewed	69.67 ± 3.36	87.49 ± 4.24	258.52 ± 8.85

Table 1: Comparison of memory management strategies during the join phase. Tested with OpenMP dynamic scheduling (chunk size 8) on $N_R = 2 \cdot 10^7$, $N_S = 2 \cdot 10^7$, $P = 256$, $n_threads = 32$, 10 run, Max-key = 1×10^7 , Node07.

As previously stated in Module 2, the reported build and probe times include only the computational part of the execution, excluding memory allocation and management overhead of the flat hash map. Under extreme skew ($\sigma = 0.95$), the Hoisted approach suffers from severe imbalance (over 740ms). While 1 Map per Partition improves upon this by distributing the data structures (414ms), the Adaptive Memory strategy proves to be the most resilient, dynamically scaling memory to the partition load to minimize structural overhead (258ms).

Local Join Kernel: Optimizing Workload Scheduling. [EXPERIMENT 1.2] After addressing the memory allocation bottleneck with the Adaptive Memory strategy, I optimized OpenMP’s scheduling parameters to improve load balancing. I benchmarked the `static`, `dynamic`, and `guided` scheduling policies with different `chunk_size` values, as well as the compiler-managed `auto` policy.

Chunk	Static (ms)		Dynamic (ms)		Guided (ms)	
	Uniform	Skewed	Uniform	Skewed	Uniform	Skewed
Def.	179.0 ± 2.3	249.2 ± 2.9	–	–	180.5 ± 2.3	253.7 ± 14.8
1	177.9 ± 2.1	251.8 ± 11.2	176.3 ± 1.4	255.8 ± 9.7	177.1 ± 1.3	250.2 ± 8.4
4	177.5 ± 1.3	245.0 ± 2.0	176.9 ± 1.1	247.9 ± 6.0	187.2 ± 4.6	257.6 ± 13.6
8	176.8 ± 1.5	262.9 ± 2.0	177.2 ± 1.9	249.7 ± 9.1	177.2 ± 0.8	249.7 ± 4.0
16	169.1 ± 2.9	363.0 ± 1.0	258.0 ± 29.2	369.9 ± 16.8	278.8 ± 21.0	362.9 ± 19.9
32	224.8 ± 17.1	387.4 ± 5.3	278.9 ± 16.4	400.0 ± 15.0	273.7 ± 22.9	400.1 ± 13.5
64	298.5 ± 1.7	520.9 ± 4.2	365.6 ± 22.8	537.6 ± 14.1	350.2 ± 20.2	541.9 ± 5.8
Auto	Uniform: 177.8 ± 1.5 ms Skewed: 243.9 ± 2.9 ms					

Table 2: the experiments were performed with $N_R = 2 \cdot 10^7$, $N_S = 2 \cdot 10^7$, $P = 256$, $n_threads = 32$, 10 run, Max-key = 1×10^7 , Node07.

The results show that large chunk sizes (≥ 16) significantly degrade performance across all scheduling policies, especially under skewed workloads. In contrast, small chunks (1–8) keep execution time stable around 175–250 ms. Notably, static scheduling

with chunk size 4 performs well in this specific skewed case (~ 245 ms) due to effective round-robin distribution of small tasks across threads, which balances hot partitions while avoiding dynamic scheduling overhead. However, for unpredictable workloads, dynamic and guided scheduling with small chunks remain more robust choices, as they do not rely on a favorable static mapping of heavy partitions to threads.

2.2 OpenMP Task-Based Parallelism

After evaluating traditional cycle-level scheduling, we now explore the alternative parallelization paradigm using explicit OpenMP tasks, as required. All three kernels follow the same task-based pattern presented in the lectures: inside a `parallel` region, a `single` thread spawns an `untied` macro-task responsible for dynamically generating the worker tasks.

Task-Centric Histogram Computation. In the loop-based version (`compute_histogram_matrix`), each thread was statically assigned a partition block; this avoided synchronizations by maintaining a 2D matrix of size `num_threads`, where each thread updated only its own row via `omp_get_thread_num()`.

With the task-based approach, this scheme is no longer valid, as the OpenMP runtime dynamically schedules tasks via *work-stealing*. There is no guaranteed 1-to-1 correspondence between a thread and a data block. If a single thread executes multiple tasks, it will accumulate the counts of different data blocks into the same thread-local row. While mathematically the total would be correct at this stage of histogram construction, this could potentially mix the histograms of different data blocks, losing block-level granularity. This granularity is strictly necessary in the subsequent prefix sum and scattering phases to assign independent memory offsets to each block. For this reason, the histogram matrix is sized with respect to `num_chunks`.

Task-based scatter analysis: block-level buffering and offsets. The scatter phase shows why block granularity established in the histogram step is necessary. In a thread-indexed scheme (as in the loop-based version), dynamic scheduling introduces data races: the same worker thread may process tasks from different blocks while reusing thread-local write cursors. For example, executing both Block 0 and Block 5 would overwrite the offset state, causing records from Block 5 to be written into the memory region of Block 0 and corrupting the partitioning.

By computing histograms and prefix-sum offsets over `num_chunks` upfront, this issue is removed. In the task-based implementation (`scatter_omp_tasks`), both buffers and write positions are strictly bound to the block index `c`: each task uses its own `local_buffer` and exclusive cursor (`my_next = chunk_offsets[c]`), ensuring full spatial determinism independent of runtime thread scheduling.

Task-based local join. In the loop-based approach, threads declared the adaptive memory (`ska::flat_hash_map`) in their local scope, making it naturally visible to the `#pragma omp for` iterations. Transitioning to a task-based model breaks this trivial scoping. Allocating a new map inside each of the dynamically scheduled tasks would cause severe overhead, completely defeating memory reuse. Thus, memory must remain bound to the executing threads. However, OpenMP tasks are isolated closures—they are completely "blind" to the local variables of the unknown worker thread that eventually dequeues them.

To bridge this visibility gap, we introduce a global pointer array (`state_ptrs`) acting as a public directory. Inside the `#pragma omp parallel` region, each thread allocates a persistent `ThreadState` on its stack and registers its address at its thread ID index. When a task executes, it queries `omp_get_thread_num()` to retrieve the corresponding `ThreadState`. This design is entirely safe because worker tasks are implicitly *tied*: once started, they never migrate across threads mid-execution. This allows tasks to safely "reconnect" with the executing thread's persistent memory, eliminating per-task allocation overhead without requiring any synchronization.

Methodological Fairness in Performance Profiling. To ensure a rigorous and unbiased comparison between the loop-based *work-sharing* and task-based *work-stealing* paradigms, our profiling methodology guarantees strict symmetry in both time measurement and memory management. For the *build* and *probe* phases, execution times are accumulated into thread-local structures to avoid synchronization overhead and subsequently reduced using `std::max()` across all threads. This accurately captures the true critical path (the makespan) of the parallel execution—completing only when the busiest thread finishes—which remains fundamentally valid regardless of how the underlying scheduler distributes the partitions. Furthermore, the task-based implementation faithfully mirrors the baseline's memory lifecycle by leveraging C++ RAII to declare local memory states directly within the `#pragma omp parallel` block for parallel allocation and destruction. Because the enclosing `ScopedTimer` captures this entire lifecycle, any observed performance delta is rigorously isolated to OpenMP's scheduling and work-stealing overhead, completely eliminating systemic bias from asymmetric memory management.

Task-based local join: Task granularity analysis. [EXPERIMENT 2] To evaluate the impact of task granularity on performance, we benchmarked the task-based join across varying `chunk_size` values for both Uniform and Skewed datasets ($N_R = N_S = 2 \times 10^7$, $p = 256$, $n_threads = 32$, 10 run, Max-key = 1×10^7 , Node07). Table 3 summarizes the median execution latencies.

Chunk Size	Uniform Dataset (ms)			Skewed Dataset (ms)		
	Build	Probe	Join Loop	Build	Probe	Join Loop
1	53.4	53.9	180.0	65.2	92.5	250.6
4	61.5	72.5	184.7	65.3	107.0	246.5
8	52.0	50.3	178.0	69.4	87.3	257.6
16	69.1	67.8	208.0	97.5	87.5	356.8
32	74.6	68.9	227.5	112.1	100.6	390.9
64	122.4	86.2	348.6	123.2	96.4	531.4
128	209.0	142.4	573.2	135.3	162.4	666.6
256	281.2	236.9	813.8	215.0	268.2	1003.2

Table 3: Median execution latencies for varying task chunk sizes.

Experimental results identify an optimal granularity window for `chunk_size` values between 1 and 8. Within this range, fine-grained tasks maximize work-stealing efficiency and maintain a balanced workload. Crucially, the overhead associated with the `flat_hash_map` (e.g., rehash and clear operations) remains minimal, as evidenced by the negligible delta between the aggregate *build-probe* time and the total *join loop* latency. Conversely, increasing `chunk_size` beyond 16 triggers monotonic performance degradation. This is driven by both reduced load-balancing flexibility and increased memory management overhead, confirming that fine-grained task distribution is essential to sustain high throughput and effectively hide the cost of persistent memory maintenance.

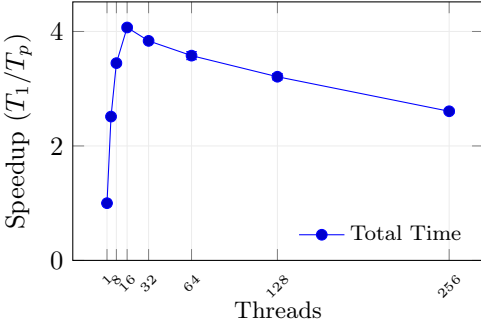
3 Scalability [EXPERIMENT 3]

3.1 Strong Scalability Analysis [EXPERIMENT 3.1]

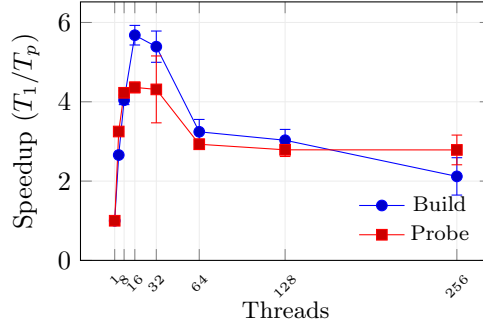
Test setup: $N_R = N_S = 20 \times 10^6$, Max Key = 10×10^6 , $P = 256$, Chunk = 16, Node07. The median time was reported, out of 10 runs. Speedup vs. Number of Threads. The Y-axis shows the ratio T_1/T_i . Error bars denote propagated uncertainty. Note: Speedup variance is calculated as $\Delta S = T_1 \cdot \frac{\Delta T_i}{T_i^2}$, treating the single-thread baseline T_1 as a constant reference.

Uniform data: Loop Level Solution

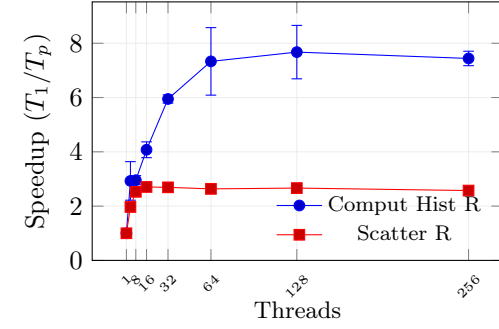
General Total Time



Local Join Phase

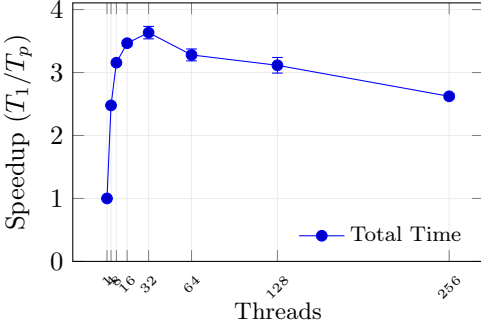


Relation R

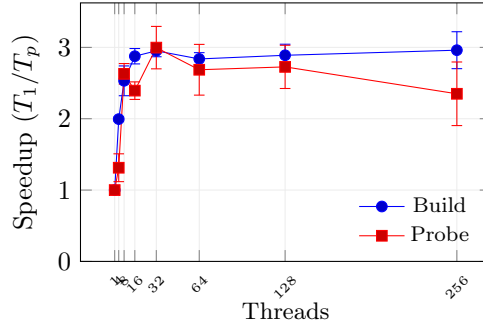


Skewed data: Loop Level Solution

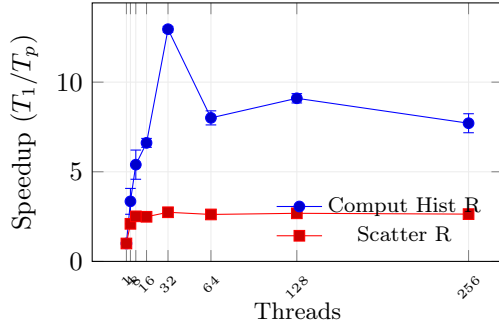
General Total Time



Local Join Phase

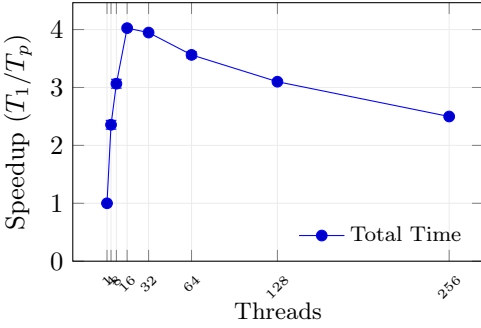


Relation R

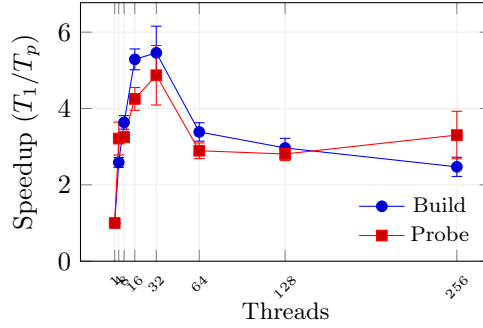


Uniform data: Task Level Solution

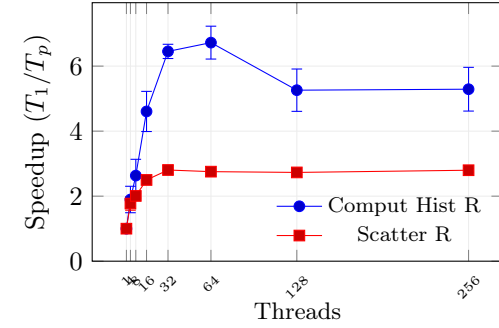
General Total Time



Local Join Phase

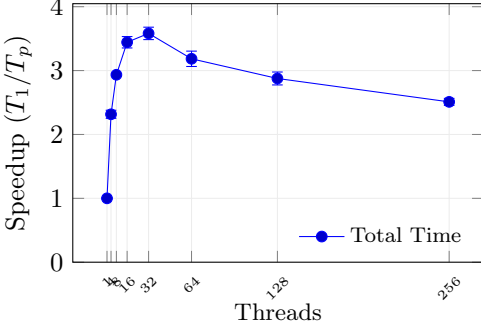


Relation R

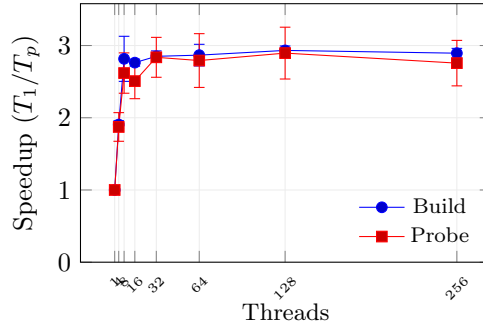


Skewed data: Task Level Solution

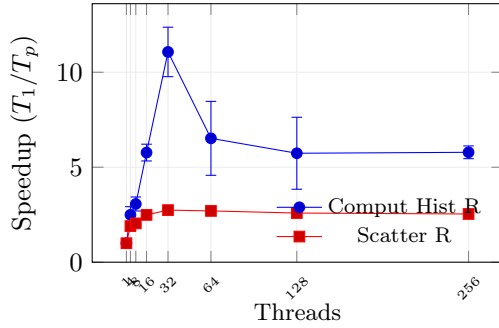
General Total Time



Local Join Phase



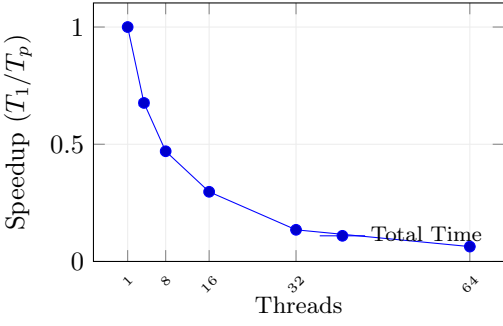
Relation R



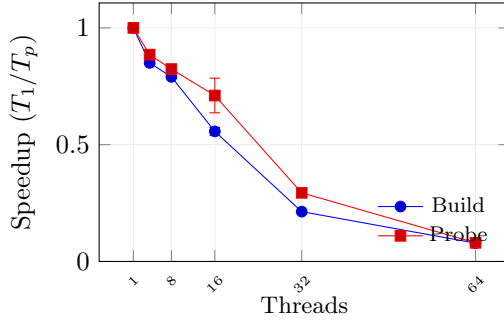
3.1 Weak Scalability Analysis [EXPERIMENT 3.2]

The weak scaling analysis on unbalanced datasets was conducted by linearly increasing the overall data volume ($N \propto t$) while maintaining the original statistical distribution of hotspots unchanged. To make the weak scaling flat, I could also scale the number of hotspots, but this would have distorted the results as it would have changed the problem instance, leading to better results.

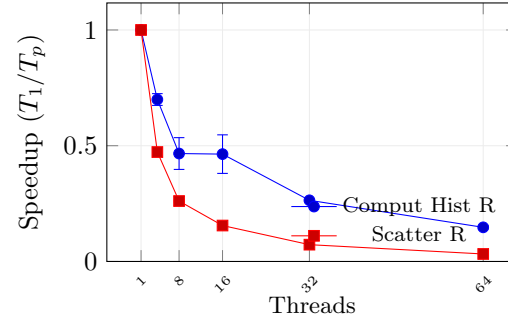
Uniform data: Loop Level Solution
General Total Time



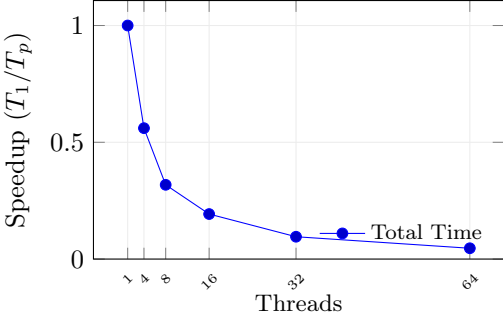
Local Join Phase



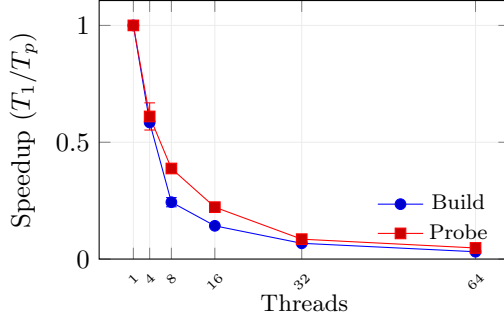
Relation R



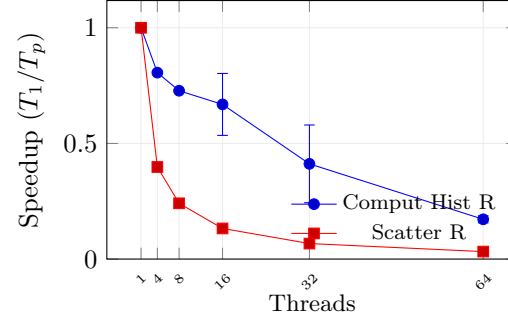
Skewed data: Loop Level Solution
General Total Time



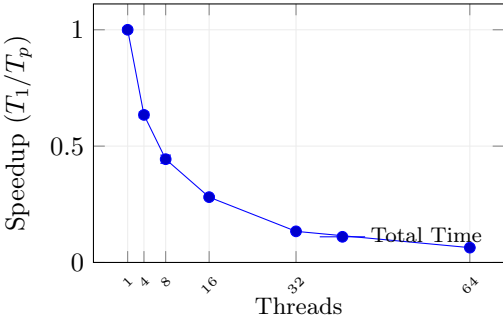
Local Join Phase



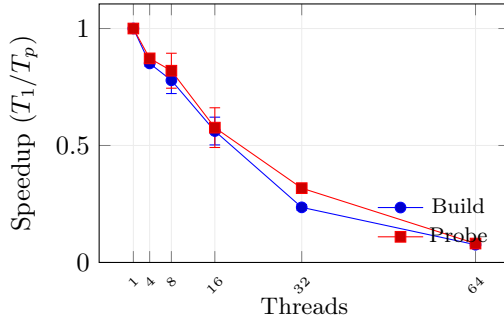
Relation R



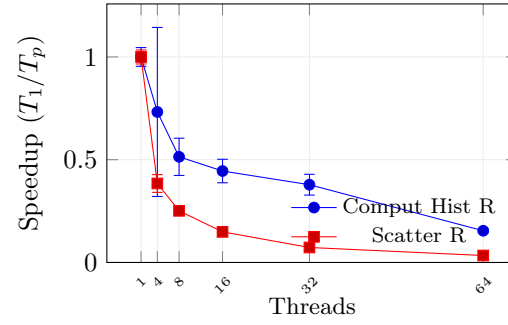
Uniform data: Task Level Solution
General Total Time



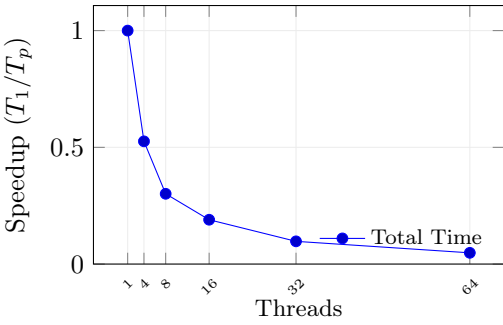
Local Join Phase



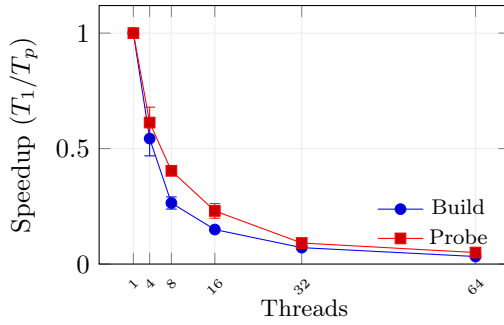
Relation R



Skewed data: Task Level Solution
General Total Time



Local Join Phase



Relation R

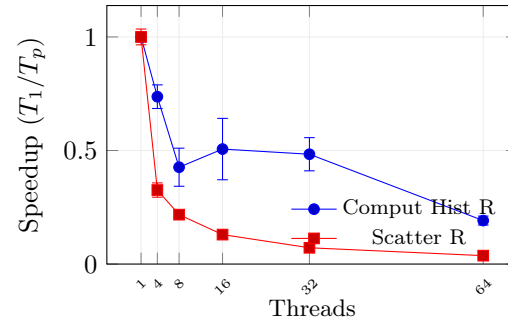


Table 1: Performance Comparison and Speedup vs Sequential Baseline. Times are in milliseconds (Median $\pm$ Std). [node07, 32 threads, $N_R = N_S = 2 \times 10^7$, $K_{max} = 10^7$, $P = 256$, seed 42, 10 run] Uniform: $\sigma = 0$ | Skewed: $\sigma = 0.95$, subset=10, $\theta = 2$

Implementation	HistR	ScatR	HistS	ScatS	Build	Probe	JoinLoop	Total
Dataset 1 (Skewed Workload)								
Sequential	36.1 ± 0.2	112.5 ± 0.3	36.1 ± 0.1	112.8 ± 0.3	207.4 ± 0.3	165.0 ± 0.3	1354.9 ± 0.8	1654.2 ± 0.7
C++ Threads	6.3 ± 0.2	61.3 ± 1.0	6.3 ± 0.6	60.4 ± 1.1	122.2 ± 12.6	113.7 ± 25.8	1032.3 ± 87.4 (1.31x)	1168.1 ± 87.4 (1.42x)
OMP Loop	4.5 ± 0.2	59.2 ± 1.4	4.9 ± 0.2	57.7 ± 1.4	68.1 ± 19.3	92.1 ± 13.1	257.4 ± 24.1 (5.26x)	380.6 ± 23.4 (4.35x)
OMP Task	3.8 ± 1.1	59.4 ± 1.1	4.7 ± 0.2	59.3 ± 0.4	65.3 ± 1.1	97.2 ± 9.0	268.0 ± 11.2 (5.06x)	396.6 ± 11.6 (4.17x)
Dataset 2 (Uniform Workload)								
Sequential	36.4 ± 0.1	326.3 ± 0.5	36.2 ± 0.1	326.5 ± 0.2	259.2 ± 0.1	228.4 ± 0.1	956.9 ± 0.4	1683.7 ± 0.4
C++ Threads	6.4 ± 0.1	57.9 ± 1.0	6.9 ± 0.6	57.8 ± 1.1	59.7 ± 2.6	65.8 ± 6.9	194.4 ± 11.8 (4.92x)	325.3 ± 12.9 (5.18x)
OMP Loop	6.3 ± 0.2	59.7 ± 0.6	6.8 ± 1.0	59.0 ± 1.1	51.3 ± 4.6	49.1 ± 9.9	177.3 ± 1.7 (5.40x)	310.4 ± 1.1 (5.42x)
OMP Task	5.9 ± 0.5	59.9 ± 0.9	6.2 ± 0.8	60.7 ± 0.4	51.9 ± 0.8	46.9 ± 4.8	180.0 ± 1.3 (5.32x)	314.7 ± 1.9 (5.35x)

Performance Analysis: As can be seen from the data in the table, the native parallel implementation in C++ offers excellent performance, comparable to OpenMP under a uniform workload. However, it suffers a serious degradation in the *asymmetric*

scenario, this clearly does not surprise me at all since that implementation was done assuming uniform data. Although there is a physiological increase in times in the *Build* and *Probe* phases (due to the fact that the basic partitioning was not designed to balance extreme computational asymmetries), the real problem arises in the total *JoinLoop* time, which suffers a real performance collapse. This "explosion" in times is directly attributable to the absence of the adaptive memory management mechanism (successfully introduced in later OpenMP variants).